Meeting Analyzer

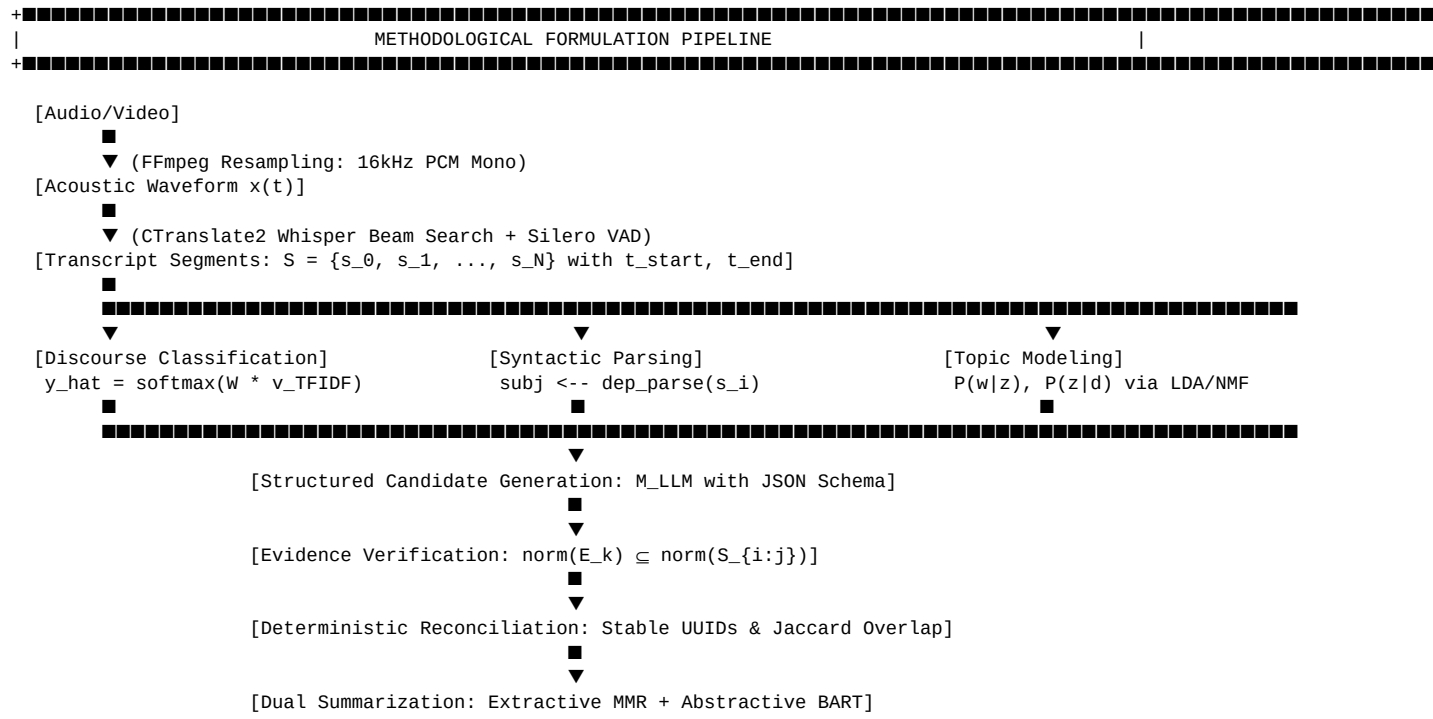
Detailed Methodology

Grounded, deterministic, and privacy-conscious meeting intelligence using hybrid NLP and structured semantic extraction

IEEE Paper Methodology Section

III. Methodology

This section describes the complete processing pipeline, mathematical formulation, evidence-grounding strategy, deterministic reconciliation engine, summarization methods, and reproducibility controls used by the proposed Meeting Analyzer system.



A. Speech-to-Text Pipeline (Faster-Whisper)

Traditional Whisper execution via PyTorch suffers from substantial memory overhead and slow decoding. We deploy Faster-Whisper, which reimplements OpenAI's Whisper model in C++ using CTranslate2. CTranslate2 applies 8-bit integer quantization (INT8) and weight-packing optimizations on CPU/CUDA, yielding a `4 x` **speedup** and a `70\%` **reduction in RAM consumption** without Word Error Rate (WER) degradation.

The model factors speech recognition as sequence-to-sequence autoregressive decoding:

```
P(Y|X) = \prod_i=1^L P(y_i \mid y_&lt;i, Encoder(X))
```

where `X` is the log-mel spectrogram extracted from the 16 kHz audio buffer, and `Y = \y_1, \dots, y_L\` is the token sequence. Timestamps are resolved at the sub-segment boundary using cross-attention matrix alignment, generating discrete time intervals `[t_start^(i), t_end^(i)]` for each segment `s_i`.

B. Discourse Act Classification

Sentences are mapped into five mutually exclusive functional discourse classes:

```
C = \ACTION ITEM, DECISION, QUESTION, DISCUSSION, INFORMATION\
```

1) Feature Representation

Given a sentence u, we compute sublinear term frequency-inverse document frequency (TF-IDF) feature vectors over word unigrams and bigrams:

```
TF-IDF(t, u, D) = (1 + \log(tf(t, u))) x \log(\frac1 + |D|1 + |\d in D : t in d\|) + 1
```

Stop words are filtered through a curated conversational stop-list that retains modal auxiliary verbs (*"will"*, *"should"*, *"must"*, *"need"*), interrogatives (*"why"*, *"when"*, *"how"*), and negation particles (*"not"*, *"never"*), which carry essential pragmatic signals.

2) Optimization Objective

We fit a multinomial Logistic Regression classifier with L_2 regularization:

$$\min_\mathbfw sum _i=1^M -\log P(y_i \mid \mathbfx_i; \mathbfw) + \frac\lambda2 \|\mathbfw\|_2^2$$

where the posterior probability is parameterized by the softmax function:

$$P(y = c \mid \mathbfx) = \frac\exp(\mathbfw_c^T \mathbfx + b_c)sum _k in C \exp(\mathbfw_k^T \mathbfx + b_k)$$

C. Dependency-Grounded Heuristic Action & Decision Extraction

To complement statistical learning, our classical engine executes syntactic dependency parsing via spaCy:

- **Action Item Extraction:** Identifies clauses containing active imperative structures or obligation modal verbs: $ROOT(v) \land (aux(v, "must"\mid"will"\mid"need") \lor mood(v) = Imperative)The grammatical subject (\text{nsubj}) is mapped to candidate entity predictions (filtered via NER\text{PERSON}labels), and the direct object (\text{dobj}) or clausal complement (\text{xcomp}$) defines the core task payload.
- **Decision Detection:** Captures explicit consensus phrases (*"we agreed to"*, *"the decision is"*, *"approved"*), incorporating negation-tree traversal to ensure that clauses like *"we decided not to launch"* are correctly classified as negative decisions rather than positive approvals.

D. Grounded Semantic Synthesis with Verifiable Citations

To extract high-level strategic takeaways without unconstrained hallucination, transcripts are processed by a high-throughput Groq LPU engine (utilizing open-weight models including `gpt-oss-120b` and `LLaMA-3-70B`) bound to a rigorous **JSON Schema**.

```
{
  "type": "object",
  "properties": {
    "action_items": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "task": { "type": "string" },
          "owner": { "type": ["string", "null"] },
          "deadline": { "type": ["string", "null"] },
          "evidence": { "type": "string" },
          "segment_ids": { "type": "array", "items": { "type": "integer" } }
        },
        "required": ["task", "evidence", "segment_ids"]
      }
    },
    "decisions": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "decision": { "type": "string" },
          "evidence": { "type": "string" },
          "segment_ids": { "type": "array", "items": { "type": "integer" } }
        },
        "required": ["decision", "evidence", "segment_ids"]
      }
    }
  },
  "required": ["action_items", "decisions"]
}
```

Verbatim Grounding Constraint

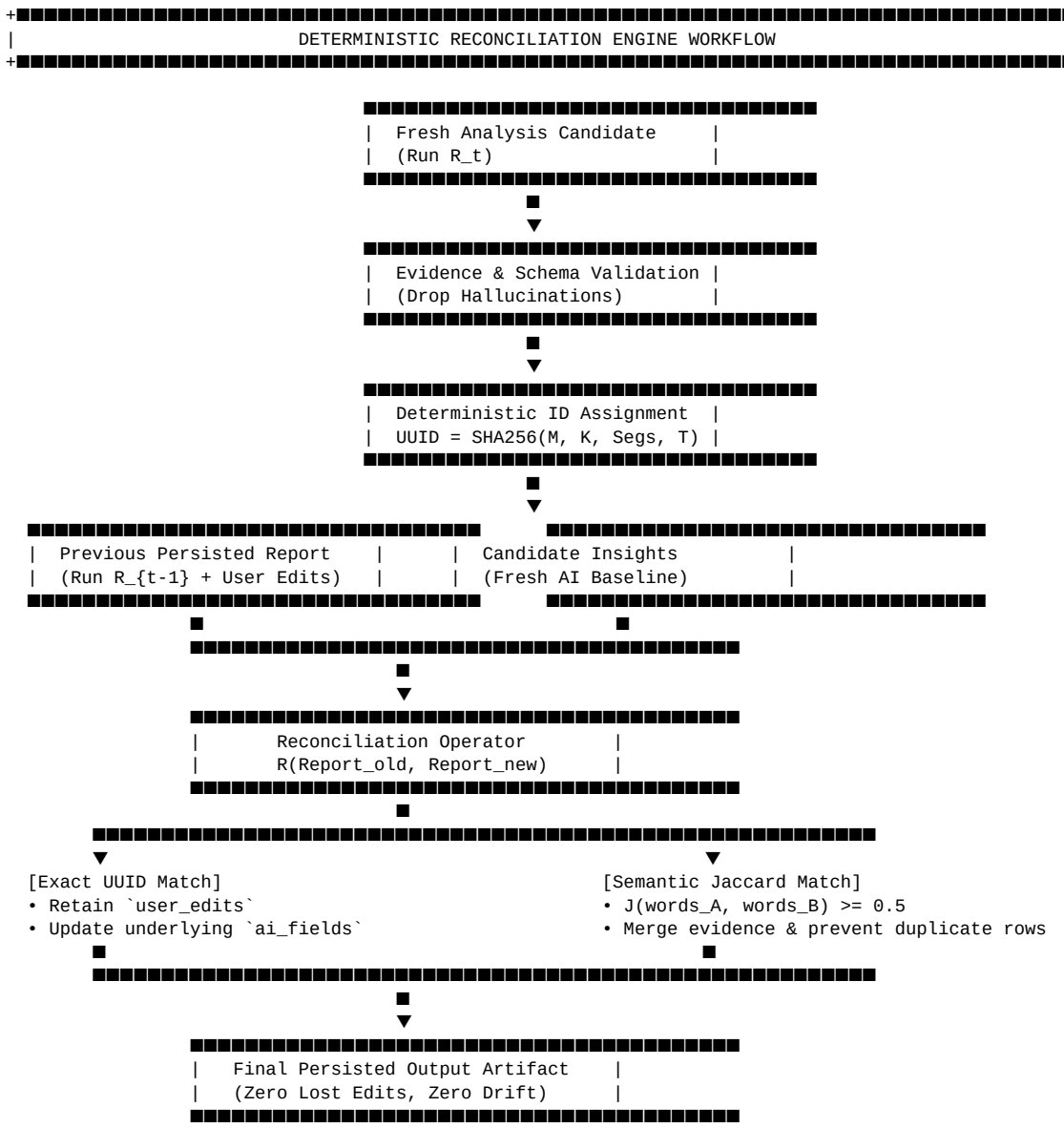
Every generated item k with evidence string E_k and cited segment sequence S_k = [i_\min, \dots, i_max] must satisfy:

$$norm(E_k) subset norm(\bigcup_j=i_\min^i_max text(s_j))$$

Any candidate failing this substring inclusion is flagged and rejected at the `evidence_matched` stage of the audit pipeline.

E. Deterministic State-Preserving Reconciliation Engine

A major contribution of this work is solving the **LLM Re-Analysis Instability Problem**. When users execute repeated analyses on the same meeting recording, standard systems generate varying counts and lose manual user edits. Our engine enforces deterministic convergence through three algorithmic stages:



1) State Manifest & Content-Derived Hashing

We define an invariant system manifest `H_state`:

```
H_input = SHA-256( Normalize(T) \parallel Lang )
H_config = SHA-256( M_name \parallel T_temp \parallel P_hash \parallel S_schema )
```

If `H_input^(t) = H_input^(t-1)` and `H_config^(t) = H_config^(t-1)`, cached analysis results are safely served unless an explicit `force = true` signal is dispatched.

2) Deterministic UUID Assignment

To prevent row instability across database re-renders, each extracted insight e receives a 24-character hexadecimal identifier derived from its categorical domain, source meeting, and cited segment sequence:

$$ID(e) = Truncate_{24}(SHA\text{-}256([MeetingID, Category, Sorted(S_k), Norm(Payload)]))$$

If two distinct tasks originate from the same dialogue turn, the normalized task string breaks ties without generating collisions.

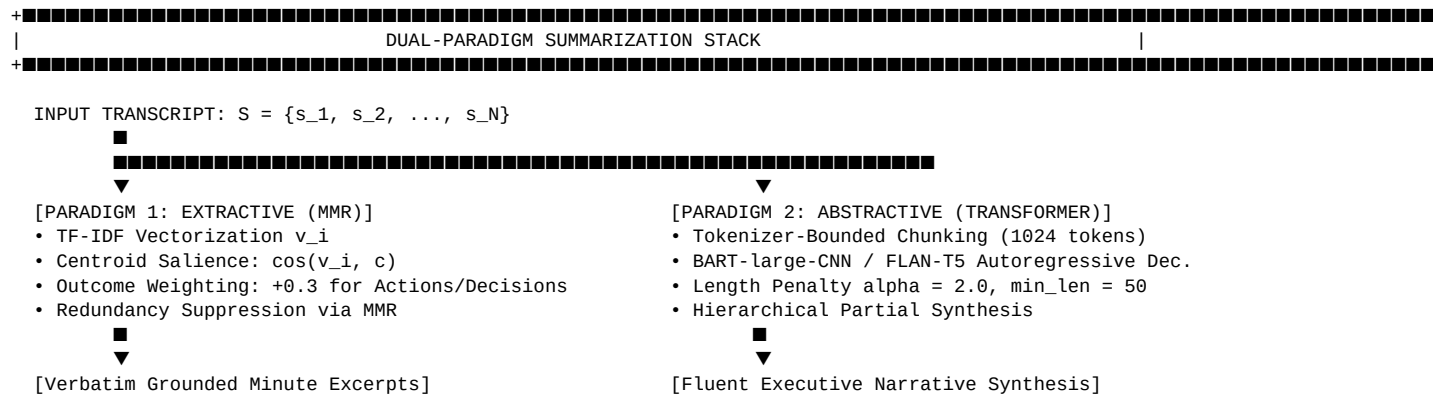
3) Human-in-the-Loop Override Matrix

Each persisted item maintains an explicit separation between AI-generated baselines and human adjustments:

$$e = id, ai_fields: \dots user_edits: \dots \}$$

When a re-analysis job executes, the reconciliation function $R(e_{old}, e_{new})$ preserves all active user edits (e.g., modified task description, manually reassigned owner, updated deadline, or marked completed status). Historical human inputs are never silently overwritten by regenerated model outputs.

F. Dual-Paradigm Summarization



1. **Extractive Mode (Maximal Marginal Relevance - MMR):** Ranks candidate transcript sentences based on similarity to the global document centroid $\mathbf{c}$ while penalizing redundancy against already selected sentences: $$MMR = \arg\max_{s_i \in S \setminus K} [\lambda \cos (\mathbf{v}_i, \mathbf{c}) - (1 - \lambda) \max_{s_j \in K} \cos (\mathbf{v}_i, \mathbf{v}_j) + \beta \cdot \mathbb{I}_{outcome}(s_i)]$$ where $\mathbb{I}_{\text{outcome}}(s_i)$ assigns additive weight (+0.30) if $sentences_i$ was classified as an Action Item or Decision.
2. **Abstractive Mode (Hugging Face Transformers):** Deploys `BART-large-CNN` or `FLAN-T5` across sliding token boundaries (1024 max input tokens) with beam search ($k=4$), length penalty $\alpha = 2.0$, and minimum summary length constraints.

G. End-to-End Processing Algorithm and Reproducibility Controls

The complete analysis procedure is implemented as an ordered, fail-closed workflow. Let an uploaded meeting be represented by the tuple $m=(f,\ell,S)$, where f is the media file, ℓ is the detected language, and S is the ordered set of timestamped transcript segments. The following procedure is applied to every meeting:

1. **Validate and normalize the input.** The upload service checks the file extension and media availability. FFmpeg removes video streams, mixes multiple channels to mono, resamples the signal to 16 kHz, and encodes signed 16-bit PCM WAV. Consequently, the ASR component receives the same acoustic representation for all supported input containers.
2. **Generate timestamped speech segments.** Faster-Whisper is loaded once per worker and executes beam-search decoding with beam size five. Voice activity detection removes pauses longer than 500 ms. For each speech segment s_i, the system stores $(i,t_i^{start},t_i^{end},x_i)$, where x_i is the original segment text. The complete transcript is the ordered whitespace-joined sequence of x_i; the original segment text is retained for evidence verification.

3 **Construct linguistic representations.** spaCy performs sentence segmentation, tokenization, part-of-speech tagging, dependency parsing, and lemmatization. A combined spaCy--NLTK stop-word set is used only for statistical features. Conversationally meaningful auxiliaries, negation, and interrogatives are preserved in classification and rule-based extraction. Filler removal is therefore not allowed to modify the evidence transcript.

4 **Run parallel classical NLP analyses.** The parsed document is passed to (i) an EntityRuler-augmented spaCy NER pipeline, (ii) the discourse-act classifier, and (iii) topic analysis. The NER stage extracts people, organizations, dates, locations, and products, while explicit patterns extend recognition to technologies and project identifiers. Topic analysis computes corpus-level TF--IDF keywords and applies LDA (or NMF when configured) with a fixed random state. These outputs are retained as interpretable intermediate artifacts rather than hidden inside the LLM prompt.

5 **Classify discourse acts.** The classifier is trained from the project-labelled sentence corpus using a persisted scikit-learn pipeline. The pipeline uses sublinear TF--IDF word n-grams (`n=1,\ldots,3`), a 20,000-feature limit, balanced class weights, and multinomial Logistic Regression. Its probability maximum is recorded as a model confidence value. High-precision action and decision rules are then applied to the prediction: explicit decision patterns take precedence, followed by obligation/assignment patterns, question markers and unresolved-issue patterns. Thus, the statistical classifier supplies broad coverage while rules protect precision for operational fields.

6 **Extract structured meeting intelligence.** For longer transcripts, the ordered segment list is partitioned into deterministic byte-bounded chunks with one-turn overlap. Each chunk is sent to the structured extraction model with the same schema and the same evidence requirements. The model must return actions, follow-ups, decisions, questions, topics, entities, summary bullets, and evidence objects. An action can contain an owner, deadline, priority, and status; missing values remain null rather than being inferred. Questions are marked resolved only when an answer and answer evidence are present.

7 **Validate evidence before persistence.** A candidate is accepted only if its evidence is a non-empty, contiguous quote in the normalized source transcript and its segment identifiers are valid integers within the transcript bounds. The quoted text must also occur in the joined span from the minimum to maximum cited segment. Invalid schema responses, incomplete output, unsupported evidence, and provider failures are rejected as analysis errors; they are never converted into an apparently successful empty report.

8 **Merge and reconcile candidates.** Chunk reports are merged locally in source order, then semantically deduplicated only when normalized task terms overlap and the evidence identifies the same event. Each item receives a stable identifier \[id(e)=\operatorname{Trunc}_{24}\left(\operatorname{SHA256}(m_{\mathrm{id}}\Vert c(e)\Vert \operatorname{sort}(\mathcal{S}_e)\Vert q(e))\right), \] where `c(e)` is the category and `q(e)` is the normalized payload used only to disambiguate distinct insights sharing a turn. Existing items are matched by source-segment overlap and normalized semantic similarity. The previous AI baseline is stored separately from `user_edits`, so edited task text, ownership, deadlines, status, and confirmation state survive re-analysis.

9 **Persist an auditable state.** The system stores the canonical input, candidate output, previous output, analysis manifest, and stage counts (`generated`, `parsed`, `validated`, `evidence_matched`, `deduplicated`, and `persisted`). The input manifest includes normalized transcript content and language. The configuration manifest includes model, prompt and schema hashes, decoding parameters, token budget, chunk policy, and analysis version. A cached report is reused only when both manifests match; an explicit re-analysis bypasses the cache but follows the same validation and reconciliation path.

The deterministic controls do not claim that a remote generative provider is mathematically deterministic. Instead, they make the application state deterministic with respect to the canonical input, versioned configuration, source evidence, and reconciliation policy. This distinction is important: provider wording may vary, but unsupported candidates are rejected, valid source-anchored items receive stable identities, and prior human edits are not silently overwritten.

H. Summary Generation and Output Contract

The default summary is extractive. After removing duplicate, very short, and routine conversational sentences, the system computes TF--IDF vectors, centroid similarity, and a redundancy penalty. Candidate sentence u_i is ranked using

```
score(u_i)= cos (\mathbfv_i,\mathbfc) -0.65max _u_j in Kcos (\mathbfv_i,\mathbfv_j) \gamma_i ,
```

where K is the selected set and \gamma_i adds weight to actions, decisions, risks, deadlines, and unresolved issues. Selected sentences are returned in their original transcript order, preserving verbatim grounding. An optional BART/T5 transformer path uses sentence-boundary chunks and is treated as a human-review draft; it does not replace the grounded extractive result when the transformer is unavailable or fails.

The final Pydantic output contract contains summary bullets, key takeaway, actions, follow-ups, decisions, questions, topics, entities, meeting metadata, and limitations. Every displayed insight retains its evidence, source segment identifiers, and—when available—the first segment timestamp. This contract is shared by the API, dashboard, and PDF exporter, preventing presentation-layer transformations from changing analytical meaning.